

Лабораторная работа №8

Оптимизация

Дурдалыев Максат

Содержание

1 Введение	5
1.1 Цели и задачи	5
2 Выполнение лабораторной работы	6
3 Вывод	17
Список литературы	18

Список иллюстраций

2.1	Линейное программирование	6
2.2	Линейное программирование	6
2.3	Векторизованные ограничения и целевая функция оптимизации . .	7
2.4	Векторизованные ограничения и целевая функция оптимизации . .	7
2.5	Оптимизация рациона питания	7
2.6	Оптимизация рациона питания	8
2.7	Оптимизация рациона питания	8
2.8	Оптимизация рациона питания	8
2.9	Оптимизация рациона питания	9
2.10	Оптимизация рациона питания	9
2.11	Путешествие по миру	9
2.12	Путешествие по миру	10
2.13	Путешествие по миру	10
2.14	Портфельные инвестиции	10
2.15	Портфельные инвестиции	11
2.16	Портфельные инвестиции	11
2.17	Портфельные инвестиции	12
2.18	Портфельные инвестиции	12
2.19	Портфельные инвестиции	12
2.20	Портфельные инвестиции	13
2.21	Восстановление изображения	13
2.22	Восстановление изображения	13
2.23	Восстановление изображения	14
2.24	Задание 1. Линейное программирование	14
2.25	Задание 1. Линейное программирование	14
2.26	Задание 2. Линейное программирование. Использование массивов .	15
2.27	Задание 2. Линейное программирование. Использование массивов .	15
2.28	Задание 3. Выпуклое программирование	15
2.29	Задание 3. Выпуклое программирование	15
2.30	Задание 4. Оптимальная рассадка по залам	16
2.31	Задание 4. Оптимальная рассадка по залам	16
2.32	Задание 5. План приготовления кофе	16

Список таблиц

1 Введение

1.1 Цели и задачи

Цель работы

Основная цель работы — освоить пакеты Julia для решения задач оптимизации[2].

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы[1].

2 Выполнение лабораторной работы

Выполним примеры из лабораторной работы (рис. [fig:001]-[fig:023]).

```
import Pkg
✓ 0.0s Julia

using JuMP
using GLPK
✓ 4.5s Julia

# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}
✓ 0.0s Julia

A JUMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

# Определение переменных x, y и граничных условий для них:
@variable(model, x >= 0)
@variable(model, y >= 0);
✓ 0.0s Julia

# Определение ограничений модели:
@constraint(model, 6x + 8y >= 180)
@constraint(model, 7x + 12y >= 128);
✓ 15s Julia
```

Рисунок 2.1: Линейное программирование

```
# Определение целевой функции:
@objective(model, Min, 12x + 20y)
✓ 1.8s Julia

12x + 20y

# Вызов функции оптимизации:
optimize!(model)
✓ 3.7s Julia

# Определение причины завершения работы оптимизатора:
termination_status(model)
✓ 2.2s Julia

OPTIMAL::TerminationStatusCode = 1

# Демонстрация первичных результирующих значений переменных x и y:
@show value(x);
@show value(y);
# Демонстрация результата оптимизации:
@show objective_value(model);
✓ 3.3s Julia

value(x) = 14.999999999999993
value(y) = 1.25000000000000047
objective_value(model) = 285.0
```

Рисунок 2.2: Линейное программирование

```
8.2.2

# Определение объекта модели с именем vector_model:
vector_model = Model(GLPK.Optimizer)
✓ 0.0s Julia

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ names registered in the model: none

# Определение начальных данных:
A = [ 1 1 9 5;
      2 5 0 0;
      2 0 6 13]
b = [7; 3; 5]
c = [1; 3; 5; 2]
✓ 3.3s Julia

4-element Vector{Int64}:
 1
 3
 5
 2

# Определение вектора переменных:
@variable(vector_model, x[1:4] >= 0)
✓ 2.9s Julia

4-element Vector{VariableRef}:
 x[1]
 x[2]
 x[3]
 x[4]
```

Рисунок 2.3: Векторизованные ограничения и целевая функция оптимизации

```
# Определение ограничений модели:
@constraint(vector_model, A * x .== b)
✓ 3.9s Julia

3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
 x[1] + x[2] + 9 x[3] + 5 x[4] == 7
 3 x[1] + 5 x[2] + 0 x[3] + 0 x[4] == 3
 2 x[1] + 6 x[3] + 13 x[4] == 5

# Определение целевой функции:
@objective(vector_model, Min, c' * x)
✓ 0.1s Julia

2x1 + 3x2 + 5x3 + 2x4

# Вызов функции оптимизации:
optimize!(vector_model)
✓ 0.1s Julia

# Определение причины завершения работы оптимизатора:
termination_status(vector_model)
✓ 0.0s Julia

OPTIMAL::TerminationStatusCode = 1

# Демонстрация результата оптимизации:
@show objective_value(vector_model);
✓ 0.0s Julia

objective_value(vector_model) = 4.9238769238769225
```

Рисунок 2.4: Векторизованные ограничения и целевая функция оптимизации

```
8.2.3

# Контейнер для хранения данных об ограничениях на количество потребляемых калорий, белков, жиров и соли:
category_data = JuMP.Containers.DenseAxisArray{ [1800 2200;
 91.0 Inf;
 0 65;
 0 1779],
 ["calories", "protein", "fat", "sodium"],
 ["min", "max"]}
✓ 3.8s Julia

2-dimensional DenseAxisArray{Float64,2,...} with index sets:
 Dimension 1, ["calories", "protein", "fat", "sodium"]
 Dimension 2, ["min", "max"]
And data, a 4x2 Matrix{Float64}:
1800.0 2200.0
 91.0   Inf
 0.0   65.0
 0.0  1779.0

# Массив данных с наименованиями продуктов:
Foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
✓ 0.7s Julia

9-element Vector{String}:
 "hamburger"
 "chicken"
 "hot dog"
 "fries"
 "macaroni"
 "pizza"
 "salad"
 "milk"
 "ice cream"
```

Рисунок 2.5: Оптимизация рациона питания

```
# Массив стоимостей продуктов:
cost = JuMP.Containers.DenseAxisArray{
    [2.49, 2.89, 1.59, 1.89, 2.89, 1.99, 2.49, 0.89, 1.59],
    Foods}

✓ 0.3s Julia

1-dimensional DenseAxisArray{Float64,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
 2.49
 2.89
 1.5
 1.89
 2.89
 1.99
 2.49
 0.89
 1.59
```

Рисунок 2.6: Оптимизация рациона питания

```
# Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:
Food_data = JuMP.Containers.DenseAxisArray{
    [410 24 26 730;
     420 32 10 1190;
     560 10 32 1800;
     380 4 19 270;
     320 12 10 930;
     320 15 12 820;
     320 31 12 1230;
     100 8 2.5 125;
     330 8 10 180],
    Foods,
    ["calories", "protein", "fat", "sodium"]}

✓ 0.3s Julia

2-dimensional DenseAxisArray{Float64,2,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
  Dimension 2, ["calories", "protein", "fat", "sodium"]
And data, a 9x4 Matrix{Float64}:
 410.0  24.0  26.0  730.0
 420.0  32.0  10.0  1190.0
 560.0  10.0  32.0  1800.0
 380.0   4.0  19.0  270.0
 320.0  12.0  10.0  930.0
 320.0  15.0  12.0  820.0
 320.0  31.0  12.0  1230.0
 100.0   8.0   2.5  125.0
 330.0   8.0  10.0  180.0
```

Рисунок 2.7: Оптимизация рациона питания

```
# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}

✓ 0.0s Julia

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

# Определение массива:
categories = ["calories", "protein", "fat", "sodium"]

✓ 0.0s Julia

4-element Vector{String}:
 "calories"
 "protein"
 "fat"
 "sodium"

# Определение переменных:
@variables(model, begin
    category_data[c, "min"] <= nutrition[c == categories] <= category_data[c, "max"]
# Сколько покушать продуктов:
    buy[Foods] >= 0
end)

✓ 2.0s Julia

(1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
  Dimension 1, ["calories", "protein", "fat", "sodium"]
And data, a 4-element Vector{VariableRef}:
 nutrition[calories]
 nutrition[protein]
 nutrition[fat]
 nutrition[sodium], 1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:
  Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
```

Рисунок 2.8: Оптимизация рациона питания


```
# Определенная целевая функция:
@objective(model, Min, sum(cost[f] * buy[f] for f in foods))
✓ 0.3s Julia

2.49bu[hamburger] + 2.89bu[chicken] + 1.5bu[hotdog] + 1.89bu[fries] + 2.09bu[macaroni] + 1.99bu[pizza] + 2.49bu[salad] + 0.89bu[milk] + 1.59bu[ice cream]

# Определенные ограничения модели:
@constraint(model, [c in categories],
sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])
✓ 1.1s Julia

1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}}, MathOptInterface.EqualTo{Float64}}, Scale
Dimension 1, ["calories", "protein", "fat", "sodium"]}
And data, a 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}}, MathOptInterface.EqualTo{Float64}}}, Scale
-nutrition[calories] + 418 buy[hamburger] + 428 buy[chicken] + 560 buy[hot dog] + 388 buy[fries] + 328 buy[macaroni] + 328 buy[pizza] + 328 buy[salad] + 180 buy[milk] + 33
-nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 28 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza] + 31 buy[salad] + 8 buy[milk] + 8 buy[ice cre
-nutrition[fat] + 26 buy[hamburger] + 18 buy[chicken] + 32 buy[hot dog] + 19 buy[fries] + 10 buy[macaroni] + 12 buy[pizza] + 12 buy[salad] + 2.5 buy[milk] + 18 buy[ice cre
-nutrition[sodium] + 738 buy[hamburger] + 1190 buy[chicken] + 1880 buy[hot dog] + 270 buy[fries] + 938 buy[macaroni] + 828 buy[pizza] + 1130 buy[salad] + 125 buy[milk] + 1
4
Generate Code Markdown

# Базовые функции оптимизатора:
JuMP.optimize!(model)
term_status = JuMP.termination_status(model)
✓ 0.1s Julia

OPTIMAL::TerminationStatusCode = 1
```

Рисунок 2.9: Оптимизация рациона питания

```
heat(buy.data, JuMP.value.(buy.data))
✓ 1.9s Julia

9x2 Matrix{AffExpr}:
buy[hamburger]  0.6045138888888888
buy[chicken]    0
buy[hot dog]    0
buy[fries]      0
buy[macaroni]   0
buy[pizza]      0
buy[salad]      0
buy[milk]       6.970138888888889
buy[ice cream]  2.591319444444444
```

Рисунок 2.10: Оптимизация рациона питания

```
8.2.4

using DelimitedFiles
using CSV
✓ 1.3s Julia

# Чтение данных:
passportdata = readlm(joinpath("passport-index-dataset", "passport-index-matrix.csv"), ',')
✓ 2.2s Julia

200x200 Matrix{Any}:
"Passport"      "Albania"      -   "Afghanistan"
"Afghanistan"   "visa required" -1
"Albania"       -1             "visa required"
"Algeria"       "visa required" "visa required"
"Andorra"       90             "visa required"
"Angola"        "visa required" -   "visa required"
"Antigua and Barbuda" 90             "visa required"
"Argentina"     90             "visa required"
"Armenia"       90             "visa required"
"Australia"     90             "visa required"
|
"Uruguay"       90             "visa required"
"Uzbekistan"    "visa required" "visa required"
"Vanuatu"       "visa required" "visa required"
"Vatican"       90             "visa required"
"Venezuela"     90             "visa required"
"Vietnam"       "visa required" "visa required"
"Yemen"         "visa required" "visa required"
"Zambia"        "visa required" "visa required"
"Zimbabwe"     "visa required" "visa required"
```

Рисунок 2.11: Путешествие по миру

```
# Задаём переменные:
cntr = passportdata[2:end,1]
vf = (x -> typeof(x)==Int64 || x == "VF" || x == "VDA" ? 1 : 0).(passportdata[2:end,2:end]);

# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}

# Переменные, ограничения и целевая функция:
@variable(model, pass[1:length(cntr)], Bin)
@constraint(model, [j=1:length(cntr)], sum( vf[i,j]*pass[i] for i in 1:length(cntr)) >= 1)
@objective(model, Min, sum(pass))

# Вывод функции оптимизации:
JuMP.optimize!(model)
termination_status(model)

# Просмотр результата:
print(JuMP.objective_value(model), " passports: ",join(cntr[findall(JuMP.value.(pass) .== 1)],",", ""))

63. 0 passports: Afghanistan, Andorra, Argentina, Australia, Azerbaijan, Bahrain, Brunei, Cambodia, Cameroon, Canada, Chile, Colombia, Comoros, DR Congo, Djibouti, Equatoria
```

Рисунок 2.12: Путешествие по миру

```
using DataFrames
using ALX
using Plots
pyplot()
using Convex
using SCS
using Statistics

# Считаем данные и размещаем их в DataFrame:
T = DataFrame{XLSX.readtable("stock_prices.xlsx","Sheet2"))

13x3 DataFrame
Row MSFT FB AAPL
Any Any Any
1 101.93 137.95 148.26
2 102.8 143.8 152.29
3 107.71 150.04 156.82
4 107.17 149.01 157.76
5 102.78 165.71 166.52
6 105.67 167.33 170.41
7 108.22 162.5 170.42
8 110.97 161.89 172.97
9 112.53 162.28 174.97
10 110.51 169.6 172.91
11 115.91 165.98 186.12
12 117.05 164.34 191.05
13 117.94 166.69 189.95
```

Рисунок 2.13: Путешествие по миру

```
using DataFrames
using ALX
using Plots
pyplot()
using Convex
using SCS
using Statistics

# Считаем данные и размещаем их в DataFrame:
T = DataFrame{XLSX.readtable("stock_prices.xlsx","Sheet2"))

13x3 DataFrame
Row MSFT FB AAPL
Any Any Any
1 101.93 137.95 148.26
2 102.8 143.8 152.29
3 107.71 150.04 156.82
4 107.17 149.01 157.76
5 102.78 165.71 166.52
6 105.67 167.33 170.41
7 108.22 162.5 170.42
8 110.97 161.89 172.97
9 112.53 162.28 174.97
10 110.51 169.6 172.91
11 115.91 165.98 186.12
12 117.05 164.34 191.05
13 117.94 166.69 189.95
```

Рисунок 2.14: Портфельные инвестиции

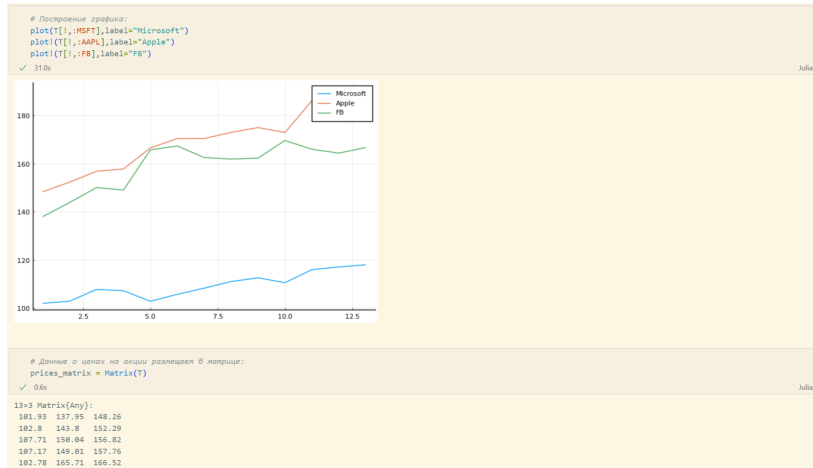


Рисунок 2.15: Портфельные инвестиции

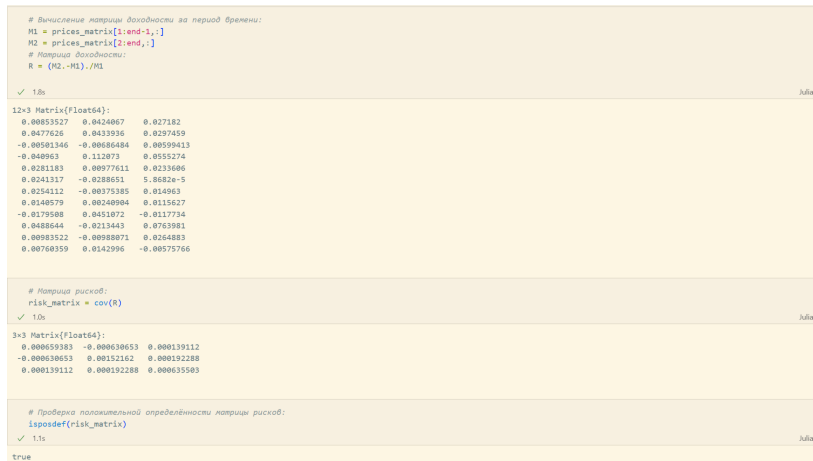


Рисунок 2.16: Портфельные инвестиции

```
# Доход от каждой из компаний:
r = mean(w,dims=1)[1:]
✓ 13s

3-element Vector{Float64}:
 0.8125274878516572
 0.81656383685529373
 0.82114588465583291

# Вектор неизвестных:
x = Variable{length(r)}
✓ 0.8s

Variable
size: (3, 1)
sign: real
convexity: affine
id: 151.465

# Объявляем модель:
problem = minimize(Convex.quadform(x,risk_matrix),[sum(x)==1;r'*x>=0.02;x.>=0])
✓ 26.1s

Problem statistics
problem is DCP      : true
number of variables : 1 (3 scalar elements)
number of constraints : 5 (5 scalar elements)
number of coefficients : 19
number of atoms      : 14

Solution summary
termination status : OPTIMIZE_NOT_CALLED
primal status      : NO_SOLUTION
dual status         : NO_SOLUTION

Expression graph
minimize
```

Рисунок 2.17: Портфельные инвестиции

```
# Найдено решение:
solve!(problem, SCS.Optimizer)
✓ 344s

Info: [Convex.jl] Compilation finished: 25.81 seconds, 810.683 MiB of memory allocated
@ Convex C:\Users\aleksey\Julia\packages\Convex\IPPOptVar\src\solution.jl:187
-----
SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 5, constraints m: 12
cones:   z: primal zero / dual free vars: 1
         1: linear vars: 4
         4: soc vars: 7, qsize: 2
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
         alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
         max_iters: 100000, normalize: 1, rho_x: 1.00e-06
         acceleration_lookback: 10, acceleration_interval: 10
         compiled with openmp parallelization enabled
lin-sys: sparse-direct-and-qdldl
         nnz(A): 22, nnz(P): 0
-----
iter | pri res | dual res | gap | obj | scale | time (s)
-----
0 | 1.45e+01 | 1.00e+00 | 2.81e+01 | 9.95e+00 | 1.00e-01 | 9.30e-03
75 | 1.70e-04 | 3.73e-05 | 5.25e-06 | 4.62e-04 | 1.00e-01 | 9.79e-03
-----
status: solved
timings: total: 9.88e-03s = setup: 1.75e-03s + solve: 8.05e-03s
         lin-sys: 2.59e-04s, cones: 5.56e-05s, accel: 8.80e-06s
-----
objective = 0.808862
-----

Problem statistics
problem is DCP      : true
number of variables : 1 (3 scalar elements)
number of constraints : 5 (5 scalar elements)
number of coefficients : 19
number of atoms      : 14
```

Рисунок 2.18: Портфельные инвестиции

```
sum(x.value)
✓ 0.3s
1.000002434677944

r'*x.value
✓ 17s
1x1 adjoint(::Vector{Float64}) with eltype Float64:
 0.819947233108531883

x.value .* 1000
✓ 0.1s

3x1 Matrix{Float64}:
 77.48789795831823
116.06771003983785
 886.5276268877958
```

Рисунок 2.19: Портфельные инвестиции

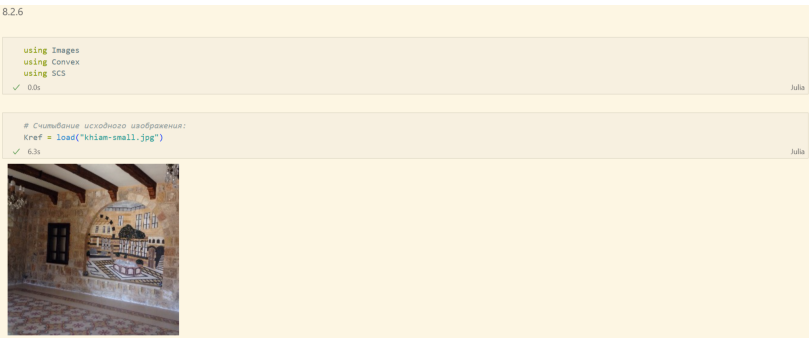


Рисунок 2.20: Портфельные инвестиции



Рисунок 2.21: Восстановление изображения



Рисунок 2.22: Восстановление изображения



Рисунок 2.23: Восстановление изображения

Теперь выполним задания для самостоятельной работы (рис. [fig:024]-[fig:032]).

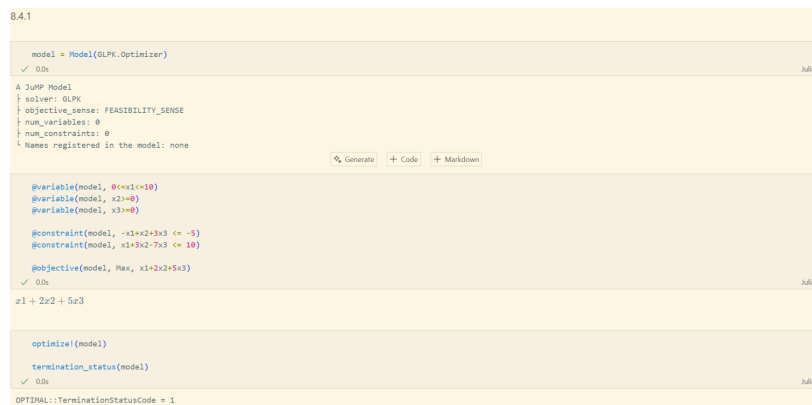


Рисунок 2.24: Задание 1. Линейное программирование

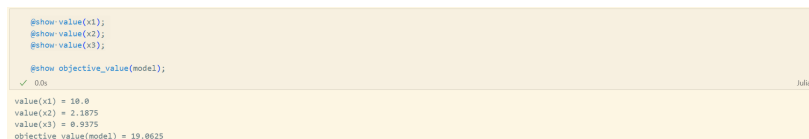


Рисунок 2.25: Задание 1. Линейное программирование

```
8.4.2

vector_model = Model{GLPK.Optimizer}
A = [-1 1 3; 1 3 -7]
b = [-5; 10]
c = [1; 2; 5]
✓ 0.0s Julia

3-element Vector{Int64}:
 1
 2
 5

@variables(vector_model, begin
  | x[1:3] = 0
  | end)
✓ 0.1s Julia

(VariableRef{X[1], X[2], X[3]},)

@constraint(vector_model, A * X .<= b)
@constraint(vector_model, X[1] <= 10)
@objective(vector_model, Min, c' * X)
✓ 0.0s Julia


$$X_1 + 2X_2 + 5X_3$$

Generate Code Markdown

optimize!(vector_model)
termination_status(model)
✓ 0.0s Julia

OPTIMAL::TerminationStatusCode = 1
```

Рисунок 2.26: Задание 2. Линейное программирование. Использование массивов

```
@show value(x1);
@show value(x2);
@show value(x3);
✓ 0.0s Julia

value(x1) = 10.0
value(x2) = 2.1875
value(x3) = 0.9375

@show objective_value(vector_model);
✓ 0.0s Julia

objective_value(vector_model) = 5.0
```

Рисунок 2.27: Задание 2. Линейное программирование. Использование массивов

```
8.4.3

n = 3
m = 4
A = rand(1:100, n, m);
b = rand(1:100, m);
✓ 0.0s Julia

x = Variable{n}
problem = minimize(sumsquares(A*x-b))
constraints = [x>=0]
solve!(problem, SCs.Optimizer)
✓ 0.0s Julia

-----
SCS v3.2.0 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 4, constraints m: 6
cones:  q: soc vars: 6, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
         alpha: 1.50, scale: 1.00e+01, adaptive_scale: 1
         max_iter: 10000, normalize: 1, rho_n: 1.00e+00
         acceleration_lookback: 10, acceleration_interval: 10
         compiled with openmp parallelization enabled
lin-sys: sparse-direct-and-qdldl
         nnz(A): 14, nnz(P): 0
-----
iter | pri res | dual res | gap | obj | scale | time (s)
-----
0 | 1.52e+03 | 1.80e+00 | 2.15e+03 | -1.87e+03 | 1.80e+01 | 4.83e-04
250 | 1.29e+00 | 1.41e-02 | 1.20e+00 | 2.10e+02 | 2.42e+00 | 1.96e-03
350 | 3.05e-05 | 8.75e-07 | 7.55e-06 | 2.10e+02 | 2.42e+00 | 2.40e-03
-----
status: solved
timings: total: 2.40e-03s = setup: 1.24e-04s + solve: 2.28e-03s
         lin-sys: 9.57e-05s, cones: 5.52e-05s, accel: 8.72e-05s
```

Рисунок 2.28: Задание 3. Выпуклое программирование

```
problem.optval
✓ 0.0s Julia

289.99693146573885
```

Рисунок 2.29: Задание 3. Выпуклое программирование

```
8.4.4

using Random
✓ 0.0s Julia

priorities = [shuffle([1, 2, 3, 10000, 10000]) for _ in 1:1000]
priorities = hcat(priorities...)
priority_weights = sum(priorities, dims = 1)
✓ 0.1s Julia

1x5 Matrix{Int64}:
3871195 4221148 3971229 4831189 3911239

model1 = Model{GLPK.Optimizer};
✓ 0.0s Julia

@variable(model1, 100<t1<250)
@variable(model1, 100<t2<250)
@variable(model1, t3 == 220)
@variable(model1, 100<t4<250)
@variable(model1, 100<t5<250);
✓ 0.0s Julia

@constraint(model1, t1+t2+t3+t4+t5==1000)
@objective(model1, Min, t1 * priority_weights[1] +
t2 * priority_weights[2] +
t3 * priority_weights[3] +
t4 * priority_weights[4] +
t5 * priority_weights[5])
✓ 0.0s Julia

3871195d1 + 4221148x2 + 3971229x3 + 4031189x4 + 3911239x5
```

Рисунок 2.30: Задание 4. Оптимальная рассадка по залам

```
optimize!(model1)
termination_status(model1)
✓ 0.0s Julia

OPTIMAL::TerminationStatusCode = 1

println("t1 = ", value(t1))
println("t2 = ", value(t2))
println("t3 = ", value(t3))
println("t4 = ", value(t4))
println("t5 = ", value(t5))
println("Значение целевой функции: ", objective_value(model1))
✓ 0.0s Julia

t1 = 240.0
t2 = 180.0
t3 = 220.0
t4 = 180.0
t5 = 180.0
Значение целевой функции: 3.99220886e9
```

Рисунок 2.31: Задание 4. Оптимальная рассадка по залам

```
8.4.5

coffee = ["Raf coffee", "Cappuccino"]
products = ["Grains", "Milk", "Sugar"]
costs = JuMP.Containers.DenseAxisArray{[400, 300], coffee}
coffee_data = JuMP.Containers.DenseAxisArray{
[40 140 5],
30 120 0,
coffee,
products
}
store = JuMP.Containers.DenseAxisArray{[500, 2000, 40], products}
coffee_data["Raf coffee", "Grains"]
store["Grains"]
✓ 0.0s Julia

500

model = Model{GLPK.Optimizer}
@variable(model, buy[coffee] >= 0)

@objective(model, Max, sum(costs[c]*buy[c] for c in coffee))
@constraint(model, [p in products], sum(coffee_data[c, p]*buy[c] for c in coffee) <= store[p])

JuMP.optimize!(model)
term_status = JuMP.termination_status(model)
hcat(buy_data, JuMP.value.(buy_data))
✓ 0.0s Julia

2x2 Matrix{AffExpr}:
buy[Raf coffee] 8
buy[Cappuccino] 6
```

Рисунок 2.32: Задание 5. План приготовления кофе

3 Вывод

В ходе выполнения лабораторной работы я освоил пакеты Julia для решения задач оптимизации.

Список литературы

- [1] *Julia 1.11 Documentation*. URL: <https://docs.julialang.org/en/v1/>.
- [2] *JuliaLang*. URL: <https://julialang.org/>.